# UE Machine Learning

## Projet Réseaux de Neurones

**Partie I :** Développement d'un Perceptron from Scratch

**Partie II :** Classification d'Images par CNN et Transfer Learning

**SYOUD Walid**

| | |
|---|---|
| **Formation :** | CNAM – Diplôme d'Ingénieur IA & Science des Données |
| **Année :** | 2023–2024 (Projet de dernière année) |
| **Langage :** | Python (Jupyter Notebook) |
| **Frameworks :** | TensorFlow, Keras, Scikit-learn, NumPy, OpenCV |

# Table des matières

**1 Introduction** **2**

**2 Partie I – Développement d'un Perceptron** **2**

2.1 Définition et Structure . . . . . 2

2.1.1 Algorithme Perceptron Simple . . . . . 3

2.2 Apprentissage Widrow-Hoff . . . . . 3

2.2.1 Principe . . . . . 3

2.2.2 Application sur deux datasets . . . . . 3

2.3 Algorithme Perceptron Multicouche (MLP) . . . . . 4

2.3.1 Architecture . . . . . 4

2.3.2 Calcul manuel de vérification . . . . . 4

2.4 Apprentissage Perceptron Multicouche – Rétropropagation . . . . . 5

2.4.1 Principe de la rétropropagation . . . . . 5

2.4.2 Application sur le problème XOR . . . . . 5

**3 Partie II – Classification d'Images** **6**

3.1 Dataset Wang . . . . . 6

3.2 Classification par Full Connected (Descripteurs) . . . . . 6

3.2.1 Descripteurs utilisés . . . . . 6

3.2.2 Architecture du MLP . . . . . 7

3.2.3 Résultats par descripteur . . . . . 7

3.3 Classification par CNN (Deep Learning) . . . . . 7

3.3.1 Progression des architectures CNN . . . . . 8

3.4 Transfer Learning avec VGG19 . . . . . 9

3.4.1 Principe . . . . . 9

3.4.2 Pourquoi VGG19 ? . . . . . 9

3.4.3 Architecture finale . . . . . 9

3.4.4 Résultats . . . . . 9

3.5 Tableau Comparatif Final . . . . . 10

**4 Conclusion** **10**

# Introduction

Dans le cadre de l'UE Machine Learning de 3ème année, ce projet explore l'un des champs d'application les plus importants du Machine Learning : les réseaux de neurones artificiels.

Les réseaux de neurones artificiels (*Artificial Neural Networks*) sont des systèmes informatiques dont le fonctionnement s'inspire des neurones du cerveau humain. Contrairement aux algorithmes classiques de ML (k-PPV, Bayes naïf, Random Forest), un réseau de neurones apprend à effectuer une tâche en analysant des exemples étiquetés, en ajustant itérativement des paramètres internes appelés **poids synaptiques**.

**Structure d'un réseau de neurones**

Un réseau neuronal est organisé en couches :

- **Couche d'entrée** : reçoit les données brutes
- **Couches cachées** (*hidden layers*) : transforment et extraient des représentations intermédiaires
- **Couche de sortie** : produit la prédiction finale
- **Poids** : valeurs numériques sur chaque connexion, modulant l'importance relative des signaux

Ce rapport se décompose en deux parties :

1. **Développement d'un perceptron from scratch** : implémentation en Python pur des algorithmes de perceptron simple, d'apprentissage Widrow-Hoff, de perceptron multicouche et de rétropropagation du gradient
2. **Classification d'images** : application sur le dataset Wang (1000 images, 10 catégories) via une approche Full Connected sur descripteurs, puis Deep Learning par CNN, et enfin Transfer Learning

# Partie I – Développement d'un Perceptron

## Définition et Structure

Un **Perceptron** est un neurone artificiel, l'unité de base des réseaux de neurones. Il s'agit d'un algorithme d'apprentissage supervisé pour la classification binaire, capable de séparer linéairement deux classes.

Pour $p$ variables d'entrée $x_1, x_2, \ldots, x_p$, la sortie du perceptron est définie par :

$$f(x) = a\left(\sum_{j=1}^{p} w_j x_j + w_0\right) \tag{1}$$

où $w_j$ sont les poids synaptiques, $w_0$ le biais, et $a$ la fonction d'activation choisie.

### Algorithme Perceptron Simple

L'objectif est d'implémenter un perceptron simple calculant la sortie pour une entrée en $R^2$.

**Paramètres de l'algorithme :**

- `w` : vecteur de poids synaptiques (3 lignes : biais + 2 poids)
- `x` : vecteur d'entrée (2 lignes)
- `active` : choix de la fonction d'activation

**Fonctions d'activation implémentées :**

- `active == 0` : $\phi(x) = \text{sign}(x)$ (signe)
- `active == 1` : $\phi(x) = \tanh(x)$ (tangente hyperbolique)

**Test sur la porte logique OU :** Le perceptron a été testé sur l'exemple classique de la porte logique OU. Les résultats obtenus correspondent exactement à la table de vérité attendue, confirmant que l'algorithme est correctement implémenté. Les deux classes (0 et 1) sont **linéairement séparables** par une droite, comme illustré par le graphe des points et la droite séparatrice associée aux poids du neurone.

**Résultat**

Le perceptron simple classifie correctement toutes les entrées de la porte logique OU. La droite séparatrice divise bien le plan en deux régions correspondant aux deux classes, validant l'implémentation.

### Apprentissage Widrow-Hoff

L'apprentissage Widrow-Hoff est une règle d'apprentissage basée sur la **descente de gradient stochastique** pour minimiser l'erreur quadratique du perceptron.

#### Principe

À chaque itération, les poids sont mis à jour selon la règle :

$$w \leftarrow w + \eta \cdot (y_{\text{attendu}} - y_{\text{prévu}}) \cdot x \tag{2}$$

où $\eta$ est le taux d'apprentissage (*learning rate*).

**Impact du taux d'apprentissage :**

- Si $\eta$ est trop **petit** : convergence très lente, nombreuses itérations nécessaires
- Si $\eta$ est trop **grand** : les poids oscillent et peuvent diverger, dépassant le minimum de la fonction d'erreur

#### Application sur deux datasets

L'algorithme a été appliqué sur deux fichiers de données (`p2_d1.txt` et `p2_d2.txt`), chacun contenant 2 classes de 25 individus en dimension 2.

| Dataset | Erreur finale (10 epochs) | Observations |
|---|---|---|
| Test 1 | $\approx 2$ | 5 points mal classés (bleus dans classe 2) |
| Test 2 | $\approx 15$ | Quasi parfait (1 point sur la frontière) |

Table 1 – Résultats de l'apprentissage Widrow-Hoff sur les deux datasets

- **Test 1** : L'erreur converge vers 0 en 10 epochs. La droite séparatrice est satisfaisante mais non optimale : 5 points bleus sont mal classés
- **Test 2** : Résultats légèrement meilleurs. Quasi tous les individus sont correctement classés, à l'exception d'un point sur la frontière de décision. Cependant, l'erreur numérique converge plus lentement (erreur $\approx 15$ à 10 epochs contre $\approx 2$ pour le test 1)

**Limite du perceptron simple**

L'apprentissage par perceptron est une technique de **séparation linéaire**. Il ne peut résoudre que des problèmes où les classes sont linéairement séparables. Pour des problèmes non linéaires (comme le XOR), il est nécessaire de passer à un réseau multicouche.

## Algorithme Perceptron Multicouche (MLP)

Le **Perceptron Multicouche** (*Multi-Layer Perceptron*) permet de résoudre des problèmes non linéairement séparables en introduisant des couches cachées entre l'entrée et la sortie.

### Architecture

Le MLP implémenté comporte :

- **Couche d'entrée** : 2 neurones ($x \in R^2$)
- **Couche cachée** : 2 neurones avec activation sigmoïde
- **Couche de sortie** : 1 neurone scalaire

**Paramètres :**

- `w1` : matrice de poids (3 lignes $\times$ 2 colonnes) pour la couche cachée
- `w2` : vecteur de poids (3 lignes) pour la couche de sortie

### Calcul manuel de vérification

Pour $x = [1, 1]^T$ avec les poids initiaux définis :

**Neurone 1 (couche cachée) :**

$$y_1 = 1 \times 2 + 1 \times (-1) - 0.5 = 0.5 \tag{3}$$
$$\phi(y_1) = \sigma(0.5) = 0.62 \tag{4}$$

**Neurone 2 (couche cachée) :**

$$y_2 = 1 \times 1 + 1 \times 0.5 + 0.5 = 2 \tag{5}$$
$$\phi(y_2) = \sigma(2) = 0.88 \tag{6}$$

**Neurone de sortie :**

$$y_3 = 0.62 \times (-1) + 0.88 \times 1 + 2 = 2.26 \tag{7}$$

$$\phi(y_3) = \sigma(2.26) \approx 0.90 \tag{8}$$

**Vérification**

Le calcul manuel donne $\approx 0.90$, en accord avec la valeur calculée par l'algorithme ($\approx 0.924$). La légère différence est due aux arrondis intermédiaires.

### Apprentissage Perceptron Multicouche – Rétropropagation

#### Principe de la rétropropagation

La **rétropropagation** (*backpropagation*) est l'algorithme central pour entraîner les réseaux multicouches. Elle calcule la contribution de chaque poids à l'erreur finale en propageant le gradient de la couche de sortie vers la couche d'entrée.

La mise à jour d'un poids $w_i$ est :

$$w_i := w_i - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_i} \tag{9}$$

où $\mathcal{L}$ est la fonction de coût (erreur quadratique moyenne) et $\frac{\partial \mathcal{L}}{\partial w_i}$ la dérivée partielle de l'erreur par rapport au poids $w_i$.

**Problème du gradient instable**

Dans les architectures multicouches profondes, le gradient tend à diminuer exponentiellement en remontant vers les couches basses (*vanishing gradient*), ou au contraire à exploser (*exploding gradient*). Cette instabilité est inhérente à la rétropropagation et constitue l'un des défis majeurs de l'apprentissage profond.

#### Application sur le problème XOR

Le XOR est un problème non linéairement séparable : aucune droite ne peut séparer les classes $(0,0) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, $(1,1) \to 0$.

**Résultats :** Après convergence, l'erreur tend vers 0 et le MLP prédit correctement les 4 entrées du XOR. Ce résultat démontre la capacité des réseaux multicouches à réaliser des séparations non linéaires, contrairement au perceptron simple.

**Conclusion Partie I**

Le perceptron simple est limité aux problèmes linéairement séparables. L'ajout de couches cachées et de la rétropropagation permet de résoudre des problèmes non linéaires complexes. Ces fondamentaux théoriques sont directement à la base des architectures profondes modernes (CNN, Transformers).

## Partie II – Classification d'Images

## Dataset Wang

Le dataset utilisé est la **base Wang**, un benchmark classique de classification d'images composé de 1000 images réparties en 10 catégories équilibrées.

| Propriété | Valeur |
|---|---|
| Nombre total d'images | 1 000 |
| Nombre de classes | 10 |
| Images par classe | 100 |
| Split train/test | 70% / 30% (CNN) ou 80% / 20% (Full Connected) |

Table 2 – Caractéristiques du dataset Wang

**Catégories :** Bus, Chevaux, Dinosaures, Éléphants, Fleurs, Montagne, Plats, Plage, Monuments, Jungle

**Prétraitement :**

- Redimensionnement des images à taille uniforme (256×256 ou 224×224 selon le modèle)
- Normalisation des valeurs de pixels en divisant par 255 (plage [0,1])
- Encodage des labels en valeurs numériques (LabelEncoder)

**Importance du tri des données**

Une attention particulière a été portée à l'ordre de rangement des images. Les images étant organisées par classe (ex : images 200-299 = Monuments), il était indispensable de remettre les numéros dans l'ordre après chargement. Cette étape de nettoyage a permis un gain de précision de **+20%** sur les résultats finaux.

## Classification par Full Connected (Descripteurs)

### Descripteurs utilisés

Plutôt que d'utiliser les pixels bruts, cette approche s'appuie sur des **descripteurs préalablement calculés** sur les images, stockés dans le fichier `WangSignatures.xlsx` :

| Descripteur | Type | Mesure |
|---|---|---|
| JCD | Couleur | Joint Composite Descriptor |
| PHOG | Forme/Gradient | Pyramid Histogram of Oriented Gradients |
| CEDD | Couleur + Texture | Color and Edge Directivity Descriptor |
| FCTH | Couleur + Texture | Fuzzy Color and Texture Histogram |
| Fuzzy Color Histogram | Couleur | Histogramme de couleur flou |

Table 3 – Descripteurs d'images utilisés (fichier WangSignatures.xlsx)

### Architecture du MLP

Un réseau fully-connected (MLP) avec 5 couches denses a été entraîné sur chaque type de descripteur :

| Couche | Neurones | Activation |
|---|---|---|
| Dense 1 | 270 | Tanh |
| Dense 2 | 90 | ReLU |
| Dense 3 | 60 | ReLU |
| Dense 4 | 30 | ReLU |
| Dense 5 (sortie) | 10 | Softmax |

Table 4 – Architecture du MLP – Classification par descripteurs

**Paramètres d'entraînement :** Adam, categorical crossentropy, 150 epochs, batch size = 100

### Résultats par descripteur

| Descripteur | Accuracy | Qualité |
|---|---|---|
| JCD | **0.86** | Très bon |
| CEDD | $\approx 0.83$ | Très bon |
| FCTH | $\approx 0.81$ | Bon |
| PHOG | $\approx 0.72$ | Correct |
| Fuzzy Color Histogram | $< 0.50$ | Insuffisant |
| Concaténation (tous) | $\approx 0.50$ | Médiocre |

Table 5 – Performance du MLP selon le descripteur utilisé

**Concaténation contre-productive**

La concaténation de tous les descripteurs n'améliore pas les résultats (accuracy $\approx 0.50$). L'écart d'échelle entre les différents types de mesures perturbe l'apprentissage. Il est préférable d'analyser chaque descripteur séparément ou de normaliser soigneusement avant concaténation.

**Conclusion Full Connected**

Le descripteur JCD obtient la meilleure accuracy (0.86). Cette approche montre l'importance du **feature engineering** : la qualité des descripteurs conditionne directement les performances du modèle. C'est l'inconvénient majeur des approches Full Connected – elles dépendent de la qualité des features fournies.

## Classification par CNN (Deep Learning)

Le CNN automatise l'extraction de features, éliminant le besoin de descripteurs manuels.

**Principe du CNN**

Les réseaux convolutifs appliquent des filtres (noyaux) qui se déplacent sur l'image et créent des **cartes de convolution** (*feature maps*). Les premières couches détectent des patterns simples (bords, textures), les couches profondes des patterns complexes (formes, objets). Les features sont apprises automatiquement par rétropropagation – c'est l'avantage fondamental du Deep Learning sur le feature engineering manuel.

### Progression des architectures CNN

**Modèle 1 – CNN Simple**

- Architecture : 1 couche Conv2D (32 filtres 3×3, ReLU) + MaxPooling + Dense(10, Softmax)
- Optimiseur : SGD (lr = 0.01), 50 epochs
- **Accuracy test : ≈ 20%**
- Observation : modèle trop simple pour capturer la complexité des images

**Modèle 2 – CNN + Early Stopping**

- Même architecture + arrêt anticipé (patience = 10, monitor = val_loss)
- Arrêt à l'itération 28 (la validation ne progressait plus)
- **Accuracy test : ≈ 71%** (+3 points vs modèle 1 simple)
- Observation : l'arrêt anticipé évite le surapprentissage mais ne compense pas l'architecture peu profonde

**Modèle 3 – CNN Profond**

- 2 couches Conv2D (32 puis 64 filtres) + Dropout + 3 couches Dense (32, 64, 128) + Early Stopping
- **Accuracy test : 75%**
- Observation : la complexification améliore les résultats. Les courbes train/validation sont proches, signe d'une bonne généralisation

**Modèle 4 – CNN + Data Augmentation** La **data augmentation** génère des variations des images à la volée pendant l'entraînement via `ImageDataGenerator` :

| Transformation | Paramètre |
|---|---|
| Rotation | ±40 |
| Translation horizontale | ±20% |
| Translation verticale | ±20% |
| Cisaillement (shear) | ±20% |
| Zoom | ±20% |
| Flip horizontal | Aléatoire |
| Remplissage | Nearest neighbor |

Table 6 – Paramètres de data augmentation

**Meilleur résultat CNN from scratch**

**Accuracy test : 83.67%** – meilleur score obtenu avec un CNN entraîné from scratch. La data augmentation a permis un gain de **+8.67 points** vs le CNN profond sans augmentation, en réduisant significativement le surapprentissage.
*Note : le temps d'entraînement passe de 25s à ≈ 1 minute par epoch.*

## Transfer Learning avec VGG19

### Principe

Le **Transfer Learning** consiste à réutiliser un modèle pré-entraîné sur une tâche similaire comme point de départ, plutôt que d'entraîner from scratch. Les poids déjà appris sur de grandes bases (ici ImageNet, 1.2 million d'images) encodent des représentations visuelles générales réutilisables.

### Pourquoi VGG19 ?

- Architecture robuste et bien documentée, pré-entraînée sur ImageNet (1000 classes)
- Largement disponible via Keras (`keras.applications.vgg19`)
- Bonne adaptabilité : il suffit de remplacer la couche de classification finale
- Les couches convolutives basses (détection de bords, textures) sont directement transférables à notre problème

### Architecture finale

| Composant | Description | Poids |
|---|---|---|
| VGG19 (conv layers) | Extraction de features | Gelés (pre-trained ImageNet) |
| Global Average Pooling | Réduction dimensionnelle | – |
| Dense (512) | Couche intermédiaire | Entraînés |
| Dense (10, Softmax) | Classification 10 classes | Entraînés |

Table 7 – Architecture Transfer Learning VGG19

**Paramètres d'entraînement :** Adam, categorical crossentropy, 25 epochs

### Résultats

**Meilleur résultat global – VGG19**

**Accuracy test : ≈ 91%** – F1-score global : **91.5%**
Toutes les classes sont prédites avec une précision ≥ 80%, à l'exception de la classe *Plage* qui reste difficile à distinguer (confusion avec Jungle et Montagne).
*Note : temps d'entraînement ≈ 200 secondes par epoch (vs 25s pour CNN simple).*

### Tableau Comparatif Final

| Approche | Modèle | Accuracy | Epochs |
|---|---|---|---|
| Full Connected | MLP sur descripteurs JCD | 86% | 150 |
| CNN from scratch | Simple (1 Conv) | ∼20% | 50 |
| CNN from scratch | + Early Stopping | ∼71% | 28 |
| CNN from scratch | Profond (2 Conv + Dropout) | 75% | 30 |
| CNN from scratch | + Data Augmentation | **83.67%** | 50 |
| Transfer Learning | VGG19 (ImageNet) | **91%** | 25 |

Table 8 – Comparaison de toutes les approches testées

## Conclusion

Ce projet a permis d'explorer l'ensemble de la chaîne des réseaux de neurones, des fondements théoriques jusqu'aux architectures profondes modernes :

**Partie I – Perceptron :**

- Le perceptron simple est un classificateur linéaire efficace pour les problèmes séparables
- L'apprentissage Widrow-Hoff (descente de gradient stochastique) converge rapidement sur des données bien séparées
- Le passage au MLP avec rétropropagation est indispensable pour les problèmes non linéaires (XOR)

**Partie II – Classification d'images :**

- L'approche par descripteurs manuels (Full Connected) est compétitive (86% avec JCD) mais dépendante de la qualité du feature engineering
- Les CNN automatisent l'extraction de features et progressent significativement avec la complexification de l'architecture et la data augmentation (83.67%)
- Le Transfer Learning (VGG19) est la stratégie la plus efficace sur de petits datasets, atteignant 91% en seulement 25 epochs – au prix d'un coût computationnel élevé (≈200s/epoch)

**Leçon principale**

Sur des datasets de taille limitée (1000 images), le Transfer Learning est systématiquement supérieur au CNN entraîné from scratch. Les représentations apprises sur ImageNet (millions d'images) sont suffisamment générales pour se transférer efficacement à des tâches de classification spécifiques.